

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

#define BUFFER_SIZE 1024
#define NUM_CHUNKS 10000

int main()
{
    int pipefd[2];
    pid_t pid;
    char buffer[BUFFER_SIZE];

    if (pipe(pipefd) == -1)
    {
        perror("pipe failed");
        return 1;
    }

    struct timespec start, end;
    clock_gettime(CLOCK_MONOTONIC, &start);

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid > 0)
    {
        close(pipefd[0]);
        for (int i = 0; i < NUM_CHUNKS; i++)
        {
            write(pipefd[1], "A", BUFFER_SIZE);
        }
        close(pipefd[1]);
        wait(NULL);

        clock_gettime(CLOCK_MONOTONIC, &end);
        double time_taken = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;
        double data_mb = (BUFFER_SIZE * NUM_CHUNKS) / (1024.0 * 1024.0);
        double throughput = data_mb / time_taken;

        printf("Parent (Producer) finished sending data.\n");
    }
}

```

```

        printf("Total Data Transferred: %.2f MB\n", data_mb);
        printf("Time Elapsed: %.6f seconds\n", time_taken);
        printf("Communication Efficiency (Throughput): %.2f MB/s\n", throughput);
    }
    else
    {
        close(pipefd[1]);
        int bytes_read;
        long total_bytes = 0;
        while ((bytes_read = read(pipefd[0], buffer, BUFFER_SIZE)) > 0)
        {
            total_bytes += bytes_read;
        }
        close(pipefd[0]);
        printf("Child (Consumer) received %ld bytes.\n", total_bytes);
        exit(0);
    }

    return 0;
}

```

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

```

```

int main()
{
    int pipefd[2];
    pid_t pid1, pid2;

    if (pipe(pipefd) == -1)
    {
        perror("pipe failed");
        return 1;
    }

    pid1 = fork();
    if (pid1 < 0)
    {
        perror("fork 1 failed");
        return 1;
    }

    if (pid1 == 0)
    {

```

```

        close(pipefd[0]);
        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[1]);
        execlp("ls", "ls", "-l", NULL);
        perror("execlp ls failed");
        exit(1);
    }

    pid2 = fork();
    if (pid2 < 0)
    {
        perror("fork 2 failed");
        return 1;
    }

    if (pid2 == 0)
    {
        close(pipefd[1]);
        dup2(pipefd[0], STDIN_FILENO);
        close(pipefd[0]);
        execlp("grep", "grep", ".c", NULL);
        perror("execlp grep failed");
        exit(1);
    }

    close(pipefd[0]);
    close(pipefd[1]);

    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    return 0;
}

```